

Trabajo Práctico: Gestión Colaborativa, Control de Versiones y Organización Empresarial (Git, GitHub y Jira)

Institución: Universidad Tecnológica Nacional - Tecnicatura Universitaria en Programación (TUP) - Modalidad a Distancia.

Cátedra: Organización Empresarial.

Cuerpo Docente: Prof. Gabriela Martínez (Titular), Prof. Mario Raúl López, Prof. Andrea Ramos, Prof. Carolina Bruno (Adjuntos).

Año Lectivo: 2026.

1. Introducción

En la ingeniería de software moderna y la organización de procesos productivos, la capacidad de registrar, revisar y coordinar cambios en un equipo distribuido es un pilar fundamental para garantizar la trazabilidad y la calidad. Según la "Guía de Uso de Git y GitHub" de la cátedra, el control de versiones deja de ser un mero aspecto técnico para convertirse en una herramienta de gestión y liderazgo, permitiendo que las organizaciones adopten metodologías ágiles y transparentes. Este trabajo integra la gestión de proyectos y el control de versiones bajo el paradigma de Aprendizaje Basado en Problemas (ABP).

2. Objetivos del Trabajo Práctico:

- **Comprender** la importancia del control de versiones en la organización y trazabilidad de proyectos profesionales.
- **Introducir** los conceptos fundamentales de Git y GitHub, diferenciando roles locales y remotos en el flujo de trabajo.
- **Aplicar** comandos fundamentales (add, commit, push, pull) para registrar y sincronizar el trabajo en una célula de desarrollo.
- **Experimentar** con flujos de trabajo colaborativos mediante el uso de ramas (*branches*) y solicitudes de extracción (*pull requests*).
- **Promover** el uso de entornos virtuales como Google Colab para la integración práctica del código y el control de versiones.
- **Relacionar** los principios de gestión empresarial con la colaboración técnica y la documentación profesional.

3. Escenario del Proyecto: La Célula de Desarrollo

La resolución de este TP se llevará a cabo bajo la modalidad de equipos de desarrollo (células ágiles) conformados por tres integrantes. El equipo debe analizar datos y generar scripts reproducibles siguiendo el caso de Hugo, Paco y Luis:

- **P1 - Líder y Organizador (Rol: Hugo):** Responsable de establecer la gobernanza del repositorio. Debe crear el repositorio central en GitHub, definir la estructura inicial de directorios (/scripts, /datos, /resultados) y redactar el archivo README.md con la visión general del proyecto.
- **P2 - Desarrollador Técnico (Rol: Paco):** Responsable de la lógica algorítmica. Debe desarrollar los scripts funcionales (en R o Python) para el análisis estadístico, asegurando que los productos se guarden en las carpetas correspondientes.

- **P3 - Revisor y QA (Rol: Luis):** Responsable de la documentación, seguridad y calidad. Debe realizar el "Peer Review" (revisión por pares), mejorar la documentación interna, asegurar la ausencia de datos sensibles y gestionar la integración final mediante el cierre de *Pull Requests*.

4. Consigna Parte I: Configuración y Gestión en Jira

Utilizando los conceptos de "Control de pendientes (Team Workflow)" y la trazabilidad organizacional, el equipo organizará la fase de planificación en Jira:

- **Configuración del Proyecto:** Crear un proyecto en Jira que represente la célula de desarrollo.
- **Gestión de Issues:** Crear al menos tres (3) tareas (Issues) asignadas específicamente a P1, P2 y P3, detallando sus responsabilidades según el punto anterior.
- **Mandato de Trazabilidad:** Para garantizar el vínculo entre gestión y código, se exige el uso de **Conventional Commits**. Cada commit en Git deberá iniciar con el ID del Issue de Jira correspondiente.
- *Ejemplo de formato:* PROY-1: Inicialización de estructura de carpetas /scripts y /datos.

5. Consigna Parte II: Implementación Técnica (Git & GitHub en Google Colab)

El flujo técnico debe ejecutarse estrictamente desde el entorno de Google Colab, respetando las limitaciones de autenticación del entorno.

5.1 Configuración de Seguridad y Autenticación

Cada integrante debe generar un **Personal Access Token (PAT) "Fine-grained"** siguiendo la ruta: Settings -> Developer settings -> Personal access tokens -> Fine-grained tokens.

- **Permisos:** Activar Contents: Read and write para el repositorio del proyecto.
- **Seguridad:** Está prohibido pegar el token en celdas de código de ejecución pública.

5.2 Inicialización del Entorno de Ingeniería

En una celda de Colab, configurar la identidad global para que los commits sean correctamente atribuidos:

```
!git config --global user.email "tu.email@ejemplo.com"
!git config --global user.name "Tu Nombre Completo"
```

5.3 Flujo de Trabajo Distribuido

- **P1 (Inicialización):** Crear el repo en GitHub con un README.md inicial. En Colab, ejecutar `git clone https://github.com/usuario/repo.git`.
- **P2 (Desarrollo):**
 - Crear rama: `git checkout -b feature/desarrollo-análisis`.
 - Asegurar la estructura de carpetas en el sistema de archivos local de Colab.
 - Escribir el script técnico y ejecutarlo.

- **Importante (Push en Colab):** Para subir los cambios, se debe inyectar el token en la URL de la siguiente manera: `!git push https://{TOKEN}@github.com/{USER}/{REPO}.git feature/desarrollo-analisis`
- **P3 (QA y Revisión):**
- Sincronizar cambios: `git pull origin feature/desarrollo-analisis`.
- Ejecutar `git branch -M main` para asegurar la consistencia del nombre de la rama principal.
- Realizar mejoras de documentación y comentarios.
- Abrir un *Pull Request (PR)* en GitHub y realizar al menos dos comentarios técnicos (hilos de discusión) antes del merge final.

6. Requerimientos de Seguridad y Buenas Prácticas

Práctica Recomendada	Acción Concreta
Protección de Credenciales	No exponer el Personal Access Token (PAT) en celdas visibles ni en el repositorio. En caso de filtración accidental, revocar inmediatamente el token desde la configuración de GitHub y generar uno nuevo.
Uso de .gitignore	Crear un archivo .gitignore para excluir archivos y carpetas innecesarias o sensibles, como <code>.ipynb checkpoints/</code> , <code>__ pycache __ /</code> y archivos de datos pesados (CSV, XLSX).
Trazabilidad de Commit	Cada comité debe comenzar con el ID del Issue de Jira correspondiente. Utilizar mensajes claros y descriptivos en tiempo presente, por ejemplo: "Agregar estructura de carpetas" o "Corregir lectura de datos".
Reproducibilidad	El código debe ejecutarse correctamente en Google Colab sin depender de rutas absolutas. Utilizar rutas relativas dentro de las carpetas <code>/scripts</code> , <code>/datos</code> y <code>/resultados</code> .
Calidad de Código	Incluir comentarios técnicos que expliquen el por qué de la lógica implementada, no solamente el qué hace cada instrucción.

7. Rúbrica de Evaluación Detallada

Criterio	Excelente (100%)	Satisfactorio (70%)	Insuficiente (0%)
----------	------------------	---------------------	-------------------

Gestión en Jira	Las tareas (Issues) están correctamente creadas, asignadas a cada rol y sus estados se actualizan durante todo el proceso de trabajo.	Las tareas están creadas pero presentan asignación incompleta o escasa actualización de estados.	No se evidencia uso de Jira para la planificación o seguimiento del proyecto.
Flujo de Git	Uso correcto del flujo de trabajo con Git: creación de ramas, commits consistentes y push exitoso utilizando PAT desde Google Colab.	Uso funcional de Git, pero con errores menores de sintaxis o flujo de ramas poco ordenado.	Errores graves en el uso de Git o imposibilidad de sincronizar el repositorio remoto.
Trazabilidad	Todos los commits en GitHub hacen referencia al Issue correspondiente de Jira mediante el formato de identificación exigido.	La vinculación es parcial; algunos commits no incluyen el ID de tarea correspondiente.	No existe relación entre la gestión de tareas y el historial de versiones.
Colaboración	El trabajo colaborativo se evidencia mediante Pull Requests con revisión por pares (Peer Review) y comentarios técnicos en los hilos de discusión.	Se utilizan Pull Requests pero sin comentarios técnicos ni interacción real entre los integrantes.	Se trabaja directamente sobre la rama principal sin revisión ni evidencia de colaboración.
Seguridad	Gestión adecuada del Personal Access Token (PAT) y presencia de archivo .gitignore correctamente configurado para proteger archivos sensibles.	Uso correcto del PAT, pero ausencia o configuración incompleta del archivo .gitignore .	Exposición de tokens, claves u otra información sensible en el repositorio.
Documentación	Repositorio con README.md profesional , estructura clara de carpetas y código ampliamente comentado que permite reproducir el proyecto.	Documentación básica; el código es comprensible pero con comentarios limitados.	Documentación inexistente o insuficiente para comprender o reproducir el trabajo.

8. Instrucciones de Entrega

La entrega consiste en un informe profesional que debe cumplir estrictamente con los estándares de rigurosidad técnica de la UTN:

- **Repositorio de GitHub:** Enlace al repositorio público (debe contener el historial completo de commits).
- **Tablero de Jira:** Enlace al proyecto compartido con el cuerpo docente.
- **Informe Final:** Archivo PDF que debe respetar las siguientes normas:
 - **Extensión:** Entre 4 y 8 páginas (Tamaño A4).
 - **Tipografía:** Texto en **Times New Roman 11 pt**.
 - **Títulos:** **12PT BOLD UPPERCASE** (Mayúsculas y negritas).
 - **Figuras/Tablas:** Deben incluir epígrafes concisos en **Arial 9 pt Italic**, centrados bajo la imagen.

9. Escenarios de Proyecto Disponibles

Con el fin de promover el aprendizaje basado en problemas y permitir que cada equipo explore un contexto de interés, los estudiantes deberán **seleccionar uno de los siguientes escenarios de proyecto** para desarrollar el trabajo práctico.

Independientemente del escenario elegido, todos los equipos deberán cumplir con los requerimientos técnicos establecidos en este documento, incluyendo la **gestión de tareas en Jira, el control de versiones con Git, la colaboración mediante GitHub y la documentación del proyecto**.

Escenario A – Análisis de Datos Climáticos

El equipo desarrollará un script que procese datos meteorológicos históricos de una ciudad.

Objetivo

Analizar un conjunto de datos climáticos para generar indicadores básicos de temperatura y precipitaciones.

Tareas sugeridas

- Importar un archivo CSV con registros de temperatura y precipitaciones.
- Calcular indicadores como:
 - temperatura promedio
 - temperatura máxima y mínima
 - promedio de precipitaciones
- Generar al menos un gráfico que represente la evolución de la temperatura en el tiempo.
- Guardar los resultados del análisis en la carpeta **/resultados**.

Escenario B – Análisis de Ventas de una Pequeña Empresa

El equipo trabajará con un conjunto de datos simulados de ventas comerciales.

Objetivo

Analizar la información de ventas para generar indicadores básicos que permitan interpretar el desempeño de la empresa.

Tareas sugeridas

- Importar un archivo de ventas con información de:
 - producto
 - cantidad vendida
 - precio
 - fecha de venta
- Calcular indicadores como:
 - ventas totales
 - producto más vendido
 - ventas por mes
- Generar un gráfico simple que represente la evolución de ventas.

Escenario C – Procesamiento Básico de Comentarios de Texto

El equipo analizará un conjunto de comentarios o reseñas escritas.

Objetivo

Realizar un procesamiento simple de texto que permita identificar patrones en los comentarios.

Tareas sugeridas

- Importar un archivo de texto o CSV con comentarios.
- Calcular la frecuencia de palabras más utilizadas.
- Identificar comentarios positivos o negativos mediante palabras clave simples.
- Generar un pequeño informe con los resultados obtenidos.

Escenario D – Estadísticas de Resultados Deportivos

El equipo analizará datos correspondientes a resultados de un campeonato deportivo.

Objetivo

Procesar los resultados de los partidos para generar estadísticas básicas del torneo.

Tareas sugeridas

- Importar un archivo con resultados de partidos.
- Calcular:
 - cantidad de partidos ganados por cada equipo
 - tabla de posiciones

- promedio de goles por partido
- Generar un gráfico comparativo de rendimiento entre equipos.

Consideraciones Generales

En todos los escenarios, los equipos deberán:

- Organizar el código dentro de la carpeta **/scripts**.
- Mantener los datos dentro de la carpeta **/datos**.
- Guardar gráficos y resultados en **/resultados**.
- Documentar el proyecto en el archivo **README.md**.
- Registrar el trabajo mediante comités que respeten el **mandato de trazabilidad con Jira**.

10. Datos de Ejemplo para el Desarrollo del Proyecto

Con el fin de facilitar el desarrollo del trabajo práctico y garantizar la reproducibilidad de los resultados, se recomienda utilizar **conjuntos de datos abiertos (Open Data)** disponibles públicamente. Estos datasets pueden descargarse en formatos compatibles como **CSV o Excel**, que pueden ser procesados fácilmente mediante Python, R o herramientas de análisis de datos.

Los siguientes recursos han sido seleccionados por su **accesibilidad, disponibilidad pública y adecuación para actividades educativas**.

10.1 Datos Climáticos Globales

Dataset con registros históricos de temperatura a nivel mundial que permiten analizar tendencias climáticas a lo largo del tiempo.

Características

- Datos de temperatura promedio global.
- Registros mensuales y anuales.
- Formato CSV.
- Datos provenientes del análisis climático GISTEMP.

Acceso

Dataset disponible en:

<https://datahub.io/core/global-temp>

Este dataset incluye series temporales de temperatura global utilizadas frecuentemente en investigación climática y educación científica. (datahub.io)

10.2 Datos Históricos de Temperatura Global

Dataset que contiene registros históricos de temperatura para múltiples países durante varias décadas.

Características

- Datos climáticos históricos desde aproximadamente 1940 hasta la actualidad.
- Registros mensuales y promedios anuales.
- Formato CSV descargable.
- Licencia de dominio público (CC0).

Acceso

Disponible en:

<https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data>

Este conjunto de datos ofrece información climática global con cobertura de múltiples países y periodos temporales extensos, lo que lo hace adecuado para análisis educativos de tendencias climáticas. (opendatabay.com)

10.3 Dataset de Ventas Simuladas

Dataset simple de ventas comerciales utilizado para ejercicios de análisis de datos.

Características

- Registros diarios de ventas.
- Columnas típicas como fecha de venta y monto de ventas.
- Formato CSV fácilmente importable.

Ejemplo de estructura

id	sales_date	sales amount
2	2024-01-01	4986
3	2024-01-02	5143

Acceso

Disponible en:

<https://gist.github.com/khanusama20/ee33c2869dd5cf3cebfd020be1ca43f6>

Este dataset contiene ejemplos simples de registros de ventas que permiten practicar cálculos de totales, promedios y análisis temporal de ventas. (gist.github.com)

10.4 Generador de Datasets Empresariales para Práctica

Plataforma que permite generar datasets simulados para análisis empresarial y ejercicios educativos.

Características

- Generación automática de datasets para práctica.
- Escenarios de negocio (marketing, ventas, recursos humanos, etc.).
- Descarga directa en formato CSV o Excel.
- Datos ficticios diseñados para aprendizaje.

Acceso

Disponible en:

<https://practicedatasets.online/>

Estos datasets están diseñados específicamente para actividades educativas y permiten simular situaciones empresariales reales para el análisis de datos. ([practice datasets.online](https://practice.datasets.online))

Recomendación para los Equipos

Cada equipo puede:

- Utilizar uno de los datasets sugeridos.
- Adaptarlo al escenario elegido.
- O utilizar otro dataset abierto, siempre que:
 - sea **de acceso público**
 - esté **correctamente citado en el informe**
 - permita **reproducir el análisis realizado**.

Los datos utilizados deberán almacenarse en la carpeta **/datos** del repositorio del proyecto.

11. Estructura Esperada del Repositorio

Para garantizar la **organización del proyecto, la reproducibilidad del análisis y la claridad en el trabajo colaborativo**, los equipos deberán respetar una estructura básica de carpetas dentro del repositorio de GitHub.

Esta estructura permite separar adecuadamente **datos, código y resultados**, siguiendo buenas prácticas habituales en proyectos de análisis de datos y desarrollo de software.

Estructura mínima requerida

El repositorio deberá presentar una organización similar a la siguiente:

```
repo-proyecto/  
├── datos/  
│   └── dataset.csv  
├── scripts/  
│   └── analisis_datos.py (o script.R)  
├── resultados/  
│   └── gráfico_resultados.png  
├── README.md  
└── .gitignore
```

Descripción de cada componente

Carpeta **/datos**

- Contiene los archivos de datos utilizados en el análisis.
- Los datasets deben almacenarse preferentemente en formato **CSV o Excel**.
- Se recomienda utilizar datasets de tamaño moderado para facilitar la clonación del repositorio.

Carpeta **/scripts**

- Contiene los programas o scripts desarrollados por el equipo.
- Los scripts pueden estar escritos en **Python o R**.
- El código debe incluir **comentarios técnicos que expliquen la lógica del análisis**.

Carpeta **/resultados**

- Debe almacenar los resultados generados por el análisis.
- Puede incluir:
 - gráficos
 - tablas generadas
 - archivos exportados por el script

Archivo **README.md**

Debe describir claramente el proyecto e incluir al menos:

- título del proyecto
- integrantes del equipo
- escenario elegido
- descripción del dataset utilizado
- instrucciones básicas para ejecutar el script

Archivo **.gitignore**

Debe configurarse para evitar subir archivos innecesarios al repositorio, por ejemplo:

- archivos temporales de Python
- checkpoints de notebooks
- archivos de gran tamaño no necesarios

Ejemplo básico:

`_pycache _/`

`.ipynb_checkpoints/`

`*.log`

Recomendación de Buenas Prácticas

Se recomienda realizar **commits frecuentes y descriptivos**, manteniendo siempre la relación entre el código desarrollado y las tareas registradas en Jira, de acuerdo con el **mandato de trazabilidad definido en este trabajo práctico**.
